

Microcontroladores – Atividade 02 – (individual ou em dupla)

Desenvolvimento de drivers para dispositivos I²C

Linguagem: C
Plataforma: STM32 utilizando HAL
Interface obrigatória: I²C

Objetivo: Desenvolver dois drivers independentes para comunicação via barramento I²C com os seguintes dispositivos:

- memória EEPROM serial **24AA256**
- sensor de temperatura **LM75**

O objetivo principal do trabalho é a construção de drivers reutilizáveis, organizados e de fácil integração em outros projetos. Os drivers devem encapsular completamente os detalhes de comunicação I²C, expondo apenas funções de alto nível.

Organização obrigatória: Cada driver deve ser implementado em arquivos específicos:

<driver>.h	Exemplo:	lm75.h	e	eeeprom24aa256.h
<driver>.c		lm75.c		eeeprom24aa256.c

Restrições: Não é permitido:

- acessar diretamente funções HAL_I2C_* fora do driver
- utilizar variáveis globais públicas
- misturar código da aplicação com código do driver
- alterar os protótipos das funções especificadas neste enunciado
- uso de bibliotecas de terceiros para a implementação dos acessos a interface I²C (é permitido o uso das funções HAL_I2C* e também a manipulação direta dos registradores envolvidos).

Driver 1 – EEPROM 24AA256

Microchip 24AA256 - Principais características

- Memória EEPROM serial com interface I²C
- Capacidade total de 256 Kbits (32 KBytes)
- Organização interna: 32768 × 8 bits
- Endereçamento interno de 16 bits
- Comunicação via barramento I²C
- Velocidade de operação de até 400 kHz (modo Fast)
- Retenção típica dos dados: superior a 200 anos
- Ciclos típicos de escrita: 1 milhão de operações
- Escrita em páginas de até 64 bytes
- Tensão típica de operação: 2,5 V a 5,5 V
- Possui pino WP (Write Protect) para proteção contra gravação
- Permite múltiplos dispositivos no mesmo barramento I²C através dos pinos A0, A1 e A2

Interface obrigatória

Protótipo	Parâmetros	Descrição
<code>void EEPROM_Init(I2C_HandleTypeDef *hi2c);</code>	- hi2c → ponteiro para a interface I ² C configurada pelo CubeMX	Inicializa internamente o driver associando-o ao periférico I ² C utilizado.
<code>HAL_StatusTypeDef EEPROM_WriteByte(uint16_t mem_addr, uint8_t data);</code>	- mem_addr → endereço interno da EEPROM - data → byte a ser gravado	Escreve um único byte na memória EEPROM
<code>HAL_StatusTypeDef EEPROM_ReadByte(uint16_t mem_addr, uint8_t *data);</code>	- mem_addr → endereço interno da EEPROM - data → ponteiro onde o byte lido será armazenado	Lê um único byte da EEPROM
<code>HAL_StatusTypeDef EEPROM_WriteBuffer(uint16_t mem_addr, uint8_t *buffer, uint16_t size);</code>	- mem_addr → endereço inicial - buffer → vetor de dados - size → quantidade de bytes	Escreve vários bytes sequencialmente na EEPROM*
<code>HAL_StatusTypeDef EEPROM_ReadBuffer(uint16_t mem_addr, uint8_t *buffer, uint16_t size);</code>	- mem_addr → endereço inicial - buffer → vetor de dados - size → quantidade de bytes	Lê múltiplos bytes sequenciais da EEPROM
<code>HAL_StatusTypeDef EEPROM_IsReady(void);</code>		Verifica se a EEPROM está pronta para nova operação após ciclo interno de gravação

*Obs. 1: O driver deve tratar corretamente a limitação de escrita por página da EEPROM.

Obs. 2: Para as funções que retornam valor os valores retornados são:

- HAL_OK
- HAL_ERROR
- HAL_BUSY
- HAL_TIMEOUT

Procedimento obrigatório de teste do driver EEPROM

Deve ser desenvolvido um procedimento de teste específico para validação do driver da EEPROM. Usando a comunicação serial (UART) o sistema deve exibir um menu que permita ao usuário realizar as seguintes operações:

1. Gravar valores conhecidos (um ou mais bytes) em diferentes posições (usuário define) da EEPROM.

2. Ler os valores gravados.

(Obs.: valores lidos e escritos devem estar no mesmo formato: decimal, hexadecimal ou ASCII)

Driver 2 – Sensor LM75

LM75/LM75A - Principais características

- Sensor digital de temperatura com interface I²C
- Medição direta em graus Celsius
- Faixa típica de medição: -55 °C até +125 °C
- Resolução típica: 0,125 °C (LM75A)
- Comunicação via barramento I²C
- Compatível com modo padrão e Fast Mode: até 400 kHz
- Possui registrador interno de temperatura
- Possui registrador de configuração
- Possui saída de alerta/sobret temperatura: pino OS (Overtemperature Shutdown)
- Permite configuração de limites de temperatura
- Baixo consumo de energia
- Conversão de temperatura realizada internamente pelo próprio sensor
- Não requer conversão A/D externa

Interface obrigatória

Protótipo	Parâmetros	Descrição
<code>void LM75_Init(I2C_HandleTypeDef *hi2c);</code>	- hi2c → ponteiro para a interface I ² C configurada pelo CubeMX	Inicializa o driver associando o periférico I ² C
<code>HAL_StatusTypeDef LM75_ReadTemperature(float *temperature);</code>	- temperature → ponteiro para variável float onde será armazenada a temperatura em graus Celsius (ex.: 25.5 °C)	Lê a temperatura atual do sensor.
<code>HAL_StatusTypeDef LM75_SetShutdown(uint8_t enable);</code>	enable 0 → modo normal 1 → shutdown	Habilita ou desabilita o modo shutdown

Obs.: Para as funções que retornam valor os valores retornados são:

- HAL_OK
- HAL_ERROR
- HAL_BUSY
- HAL_TIMEOUT

Procedimento obrigatório de teste do driver LM75

Deve ser desenvolvido um procedimento de teste específico para validação do driver do LM75.

O teste deve:

1. Ler continuamente a temperatura do sensor.
2. Exibir a temperatura no display de 7 segmentos de 4 dígitos.
3. Exibir a temperatura também pela UART usando printf.
4. Exibir mensagens de erro caso o sensor não responda.

Critérios de avaliação

Organização do driver	1,0
Funcionamento da EEPROM	3,0
Funcionamento do LM75	3,0
Procedimentos de teste	1,5
Qualidade da interface das funções	0,5
Comentários e legibilidade	1,0

Entrega: Deve ser entregue:

- código fonte completo
- arquivos .ioc

Observações finais

O aluno deve implementar todas as funções especificadas exatamente com os protótipos definidos neste enunciado.

A alteração dos nomes das funções ou dos parâmetros implicará desconto na nota.